Lab 4: The use of candidate moves in local search

Aleksander Hański, Kacper Kuźnik

Source code:
https://github.com/aleksanderhanski/evolutionary-computation/tree/main/lab4-candidate-moves

1 Description of the problem

We are given a set of nodes. Each node has x and y coordinates in a plane and a cost. The goal is to select exactly 50% of the nodes (rounded up if the number of nodes is odd) and form a Hamiltonian cycle through the selected nodes, so that the sum of the total length of the cycle and the total cost of the selected nodes is minimized.

Distances between nodes are Euclidean distances rounded mathematically to integers. The distance matrix is computed once, right after reading an instance. The optimization methods only use the distance matrix and the vector of node costs, never the coordinates.

For a solution given as an ordered list of nodes $(v_0, \ldots, v_{k-1})$:

$$f(v_0, \ldots, v_{k-1}) = \sum_{i=0}^{k-1} D[v_i][v_{(i+1) \bmod k}] + \sum_{i=0}^{k-1} cost[v_i]$$

Both instances, TSPA and TSPB, have 200 nodes, so every solution contains $k = 100$ nodes.

The goal of this lab is to make the steepest local search faster with *candidate moves*. We use the neighborhood that was the best in Lab 3: the inter-route exchange of a selected and a not selected node, together with the intra-route two-edges exchange (2-opt).

2 Pseudocode of the implemented algorithms

Notation: D is the distance matrix, $cost$ the node costs, n the number of nodes, $k = \lceil n/2 \rceil$. A solution is the cycle $P = (P_0, \ldots, P_{k-1})$ of selected nodes, indices are taken modulo k, so P_{i-1} and P_{i+1} are the neighbors of P_i in the cycle. A move improves the solution if the change of the objective function Δ is negative. A move is never applied to compute its value: Δ is computed from the edges (and node costs) that the move removes and adds.

Candidate edges

For every node u we find the $c = 10$ other nodes v with the smallest $D[u][v] + cost[v]$, the "nearest" nodes taking into account both the edge length and the cost of the node that the edge leads to. An edge u–v is a *candidate edge* if v is one of the nearest nodes of u. The lists are computed once per instance, before the local search.

Algorithm 1 Candidate lists

function CANDIDATELISTS(D, *cost*, c)
  for all nodes u **do**
    for all nodes v **do**
      $score[u][v] \leftarrow D[u][v] + cost[v]$
    end for
    $score[u][u] \leftarrow \infty$ ▷ a node is not its own neighbor
    $cand[u] \leftarrow$ the c nodes v with the smallest $score[u][v]$
  end for
  return *cand*
end function

Moves and deltas

Both moves are described by a position i in P and a second value x: a node (inter-route move) or a second position (intra-route move).

Inter-route move (inter, i, v): the selected node $u = P_i$ is replaced by the not selected node v, so the edges P_{i-1}–u–P_{i+1} become P_{i-1}–v–P_{i+1}.

Intra-route move, two-edges exchange (intra, i, j), $i < j$: the edges (P_i, P_{i+1}) and (P_j, P_{j+1}) are replaced by (P_i, P_j) and (P_{i+1}, P_{j+1}), which is done by reversing the part $P_{i+1}, \ldots, P_j$. Pairs of neighboring edges ($j = i+1$, or $i = 0$ and $j = k-1$) are skipped, exchanging them does not change the solution.

Algorithm 2 Deltas of the two moves

function INTERDELTA(D, *cost*, P, i, v)
  $p \leftarrow P_{i-1}$, $u \leftarrow P_i$, $x \leftarrow P_{i+1}$
  return $cost[v] - cost[u] + D[p][v] + D[v][x] - D[p][u] - D[u][x]$
end function

function EDGESDELTA(D, P, i, j)
  $a \leftarrow P_i$, $a' \leftarrow P_{i+1}$, $b \leftarrow P_j$, $b' \leftarrow P_{j+1}$
  return $D[a][b] + D[a'][b'] - D[a][a'] - D[b][b']$
end function

During the search we keep $pos[v]$, the position of every node v in P, with $pos[v] = -1$ if v is not selected. With it we can check in constant time whether a node is selected and where it is, and it is updated together with P by every applied move.

Algorithm 3 Applying a move

function APPLYMOVE(P, *pos*, *kind*, i, x)
  if *kind* = inter **then** ▷ x is the node v entering the solution
    $pos[P_i] \leftarrow -1$
    $P_i \leftarrow x$, $pos[x] \leftarrow i$
  else ▷ x is the position j of the second edge
    reverse $P_{i+1}, \ldots, P_x$
    for $t = i+1, \ldots, x$ **do**
      $pos[P_t] \leftarrow t$
    end for
  end if
end function

Candidate moves

A candidate move is a move that introduces at least one candidate edge to the solution. We never go through all moves to check whether they are candidate: the first loop goes over the selected nodes $u = P_i$, the second over the nearest nodes v of u, and only the moves that add the edge u–v are generated. Every such edge is added by exactly two moves:

- v **is selected** ($j = pos[v]$): two two-edges exchanges add u–v. Exchanging the edges that leave u and v, (P_i, P_{i+1}) and (P_j, P_{j+1}), adds u–v and P_{i+1}–P_{j+1}. Exchanging the edges that enter u and v, (P_{i-1}, P_i) and (P_{j-1}, P_j), adds P_{i-1}–P_{j-1} and u–v.
- v **is not selected**: v can be connected to u only by replacing one of the neighbors of u. Replacing P_{i-1} by v adds v–u, replacing P_{i+1} by v adds u–v. Exchanging u itself with v would not add the edge u–v, so it is not a candidate move for this edge.

A move can be generated twice (e.g. from u and from v if both are on each other's lists); this only means that its delta is computed twice. Per iteration there are at most $k \cdot c \cdot 2 = 2\,000$ candidate moves instead of $10\,000 + 4\,850 = 14\,850$ moves of the full neighborhood.

Algorithm 4 Generating the candidate moves

```
1: function CandidateMoves(P, pos, cand)
2:     for i = 0, ..., k − 1 do
3:         u ← P_i
4:         for all v ∈ cand[u] do
5:             j ← pos[v]
6:             if j ≥ 0 then                          ▷ v selected: two-edges exchanges adding u–v
7:                 for all (a, b) ∈ {(i, j), (i − 1 mod k, j − 1 mod k)} do
8:                     (a, b) ← (min(a, b), max(a, b))
9:                     if b − a > 1 and not (a = 0 and b = k − 1) then      ▷ skip neighboring edges
10:                        yield (intra, a, b)
11:                    end if
12:                end for
13:            else                                   ▷ v not selected: v replaces a neighbor of u
14:                yield (inter, i − 1 mod k, v)
15:                yield (inter, i + 1 mod k, v)
16:            end if
17:        end for
18:    end for
19: end function
```

Steepest local search with candidate moves

Algorithm 5 Steepest local search with candidate moves

```
1: function SteepestCandidates(D, cost, P, cand)
2:     pos[v] ← −1 for all nodes v
3:     for i = 0, ..., k − 1 do
4:         pos[P_i] ← i
5:     end for
6:     loop
7:         Δ* ← 0, m* ← none                          ▷ only improving moves can be chosen
8:         for all m = (kind, i, x) ∈ CandidateMoves(P, pos, cand) do
9:             if kind = inter then
10:                Δ ← InterDelta(D, cost, P, i, x)
11:            else
12:                Δ ← EdgesDelta(D, P, i, x)
13:            end if
14:            if Δ < Δ* then
15:                Δ* ← Δ, m* ← m
16:            end if
17:        end for
18:        if m* = none then                          ▷ no improving candidate move
19:            return P
20:        end if
21:        ApplyMove(P, pos, m*)
22:    end loop
23: end function
```

The candidate moves are generated again in every iteration from the current P and *pos*, lazily one by one, so no list of moves has to be stored or updated.

Baseline: steepest local search without candidate moves

The baseline is the steepest local search of Lab 3 with the same neighborhood (inter-route node exchange and two-edges exchange), which evaluates the whole neighborhood in every iteration. Here the inter-route move (inter, i, j) exchanges P_i with the j-th not selected node F_j, and its delta is InterDelta$(D, cost, P, i, F_j)$. Because P and F never change their lengths, the list of all moves is built once and stays valid during the whole search.

Algorithm 6 List of all moves (baseline)

```
1: function AllMoves(k, |F|)
2:     M ← [ ]
3:     for i = 0, ..., k − 1 do
4:         for j = 0, ..., |F| − 1 do
5:             append (inter, i, j) to M
6:         end for
7:     end for
8:     for i = 0, ..., k − 1 do
9:         for j = i + 1, ..., k − 1 do
10:            if j − i > 1 and not (i = 0 and j = k − 1) then        ▷ skip neighboring edges
11:                append (intra, i, j) to M
12:            end if
13:        end for
14:    end for
15:    return M
16: end function
```

Algorithm 7 Steepest local search (baseline)

```
1: function Steepest(D, cost, P)
2:     F ← nodes not in P
3:     M ← AllMoves(k, |F|)
4:     loop
5:         Δ* ← 0, m* ← none
6:         for all m = (kind, i, j) ∈ M do
7:             if kind = inter then
8:                 Δ ← InterDelta(D, cost, P, i, F_j)
9:             else
10:                Δ ← EdgesDelta(D, P, i, j)
11:            end if
12:            if Δ < Δ* then
13:                Δ* ← Δ, m* ← m
14:            end if
15:        end for
16:        if m* = none then                                          ▷ local optimum
17:            return P
18:        end if
19:        if m* is (inter, i, j) then
20:            swap P_i and F_j
21:        else                                                       ▷ m* is (intra, i, j)
22:            reverse P_{i+1}, ..., P_j
23:        end if
24:    end loop
25: end function
```

Starting solutions and the experiment

Algorithm 8 Random solution

```
1: function RandomSolution(n)
2:     return k distinct nodes drawn uniformly at random, in random order
3: end function
```

Algorithm 9 Computational experiment (for each instance)

```
1: starts ← 200 × RandomSolution(n)                    ▷ fixed seed 42, as in Labs 1 and 3
2: cand ← CandidateLists(D, cost, 10)
3: for all methods ∈ {Steepest, SteepestCandidates with cand} do
4:     for all P ∈ starts do
5:         run the method on a copy of P, measure its time
6:         check the result (k nodes, no node twice), compute its objective
7:     end for
8:     report min / avg / max of the objective and of the time
9: end for
10: for all c ∈ {5, 10, 15, 20} do                     ▷ extra experiment
11:     repeat the runs of SteepestCandidates with CandidateLists(D, cost, c)
12: end for
```

Both methods start from exactly the same 200 random solutions, which are also the starting solutions of the random-start methods in Lab 3. The baseline was run again on the same machine as the new method, so their running times can be compared directly; it gives exactly the same solutions as in Lab 3.

3 Results of the computational experiment

Each method was run 200 times on each instance, from the 200 random starting solutions. The results of the previous methods are taken from Lab 3. Cells are av (min – max), the best average in each column is in bold.

Method	TSPA	TSPB
Random	262956.1 (240892 – 289850)	206333.8 (186840 – 227175)
NN (end only)	87476.6 (84362 – 90075)	59397.3 (56007 – 63166)
NN (any position)	74306.5 (72328 – 77022)	59476.2 (52915 – 66164)
Greedy cycle	73743.7 (72590 – 75281)	59650.8 (55655 – 63734)
NN (any position) 2-regret	121360.3 (110950 – 135771)	73649.8 (68761 – 79756)
NN (any position) weighted	75549.4 (72419 – 78504)	55698.9 (50485 – 61406)
Greedy cycle 2-regret	120378.8 (112806 – 128089)	72484.8 (68150 – 77274)
Greedy cycle weighted	73353.5 (72011 – 75532)	57863.3 (52731 – 63370)
LS steepest, two-nodes, random start	90367.2 (83649 – 98214)	65817.9 (58910 – 74728)
LS greedy, two-nodes, random start	88269.0 (81824 – 94885)	63512.1 (57623 – 69006)
LS steepest, two-edges, random start	76184.6 (73961 – 80047)	52685.1 (50411 – 56416)
LS greedy, two-edges, random start	76244.7 (74219 – 80265)	52976.4 (50642 – 57138)
LS steepest, two-nodes, greedy start	73021.7 (71955 – 74861)	54292.9 (49978 – 61265)
LS greedy, two-nodes, greedy start	73021.5 (71955 – 74861)	54271.5 (49913 – 61265)
LS steepest, two-edges, greedy start	73003.8 (71955 – 74861)	53092.0 (49346 – 60354)
LS greedy, two-edges, greedy start	**72976.0 (71955 – 74861)**	53163.3 (49978 – 60506)
Baseline (LS steepest, two-edges)	76184.6 (73961 – 80047)	52685.1 (50411 – 56416)
LS steepest, candidate moves	80258.1 (76094 – 84534)	**52676.1 (50397 – 56560)**

Table 1: Objective function.

Running times are in milliseconds per solution, measured with `time.perf_counter`. For the local search methods only the local search is timed, without building the starting solution. The times of the previous methods come from the Lab 3 run; the baseline and the candidate moves were measured in the same run.

Method	TSPA	TSPB
Random	0.03 (0.02 – 0.09)	0.03 (0.02 – 0.10)
NN (end only)	0.34 (0.32 – 0.58)	0.36 (0.32 – 0.74)
NN (any position)	21.55 (20.27 – 38.83)	21.59 (20.28 – 32.53)
Greedy cycle	22.57 (20.41 – 38.46)	23.47 (20.32 – 40.25)
NN (any position) 2-regret	64.16 (60.22 – 91.06)	64.37 (60.21 – 86.56)
NN (any position) weighted	62.86 (60.01 – 75.05)	63.48 (59.97 – 76.70)
Greedy cycle 2-regret	61.21 (59.78 – 74.62)	62.32 (59.74 – 84.68)
Greedy cycle weighted	61.05 (59.46 – 63.09)	61.56 (59.48 – 79.57)
LS steepest, two-nodes, random start	919.42 (716.86 – 1334.12)	935.17 (735.92 – 1273.12)
LS greedy, two-nodes, random start	171.70 (107.96 – 354.11)	169.13 (126.64 – 255.85)
LS steepest, two-edges, random start	679.65 (559.77 – 867.08)	710.71 (590.09 – 938.64)
LS greedy, two-edges, random start	129.22 (110.21 – 165.52)	136.95 (108.84 – 257.82)
LS steepest, two-nodes, greedy start	36.15 (9.54 – 106.35)	102.41 (10.10 – 325.08)
LS greedy, two-nodes, greedy start	29.80 (12.83 – 79.43)	42.85 (12.38 – 93.68)
LS steepest, two-edges, greedy start	32.77 (8.96 – 96.02)	99.84 (27.03 – 231.30)
LS greedy, two-edges, greedy start	26.02 (11.07 – 109.80)	43.03 (18.92 – 80.65)
Baseline (LS steepest, two-edges)	726.96 (650.18 – 921.77)	724.35 (642.08 – 845.78)
LS steepest, candidate moves	142.66 (126.35 – 168.73)	145.39 (125.85 – 166.59)

Table 2: Running time [ms].

Number of iterations and evaluated moves

Average number of applied moves (iterations) and of evaluated moves (computed deltas) per run, including the last iteration which confirms the local optimum.

Method	TSPA iterations	TSPA evaluated	TSPB iterations	TSPB evaluated
Baseline: LS steepest, two-edges	132.7	1 985 742	132.9	1 988 564
LS steepest, candidate moves	125.4	227 642	131.7	234 178

Number of candidate neighbors

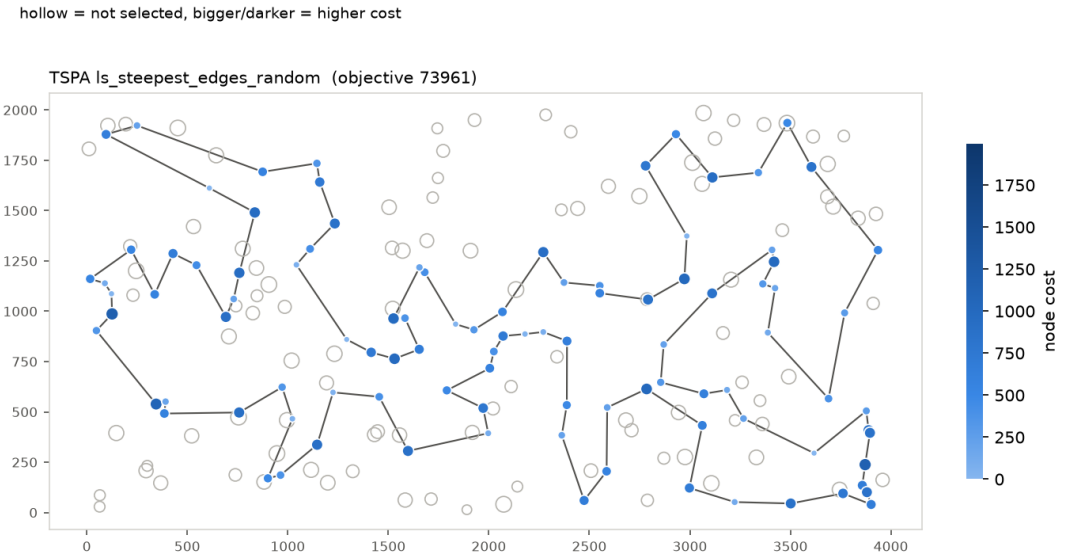
The same 200 runs of the steepest local search with candidate moves, with $c = 5, 10, 15, 20$ nearest nodes per node. Objective av (min – max) and the average time per run.

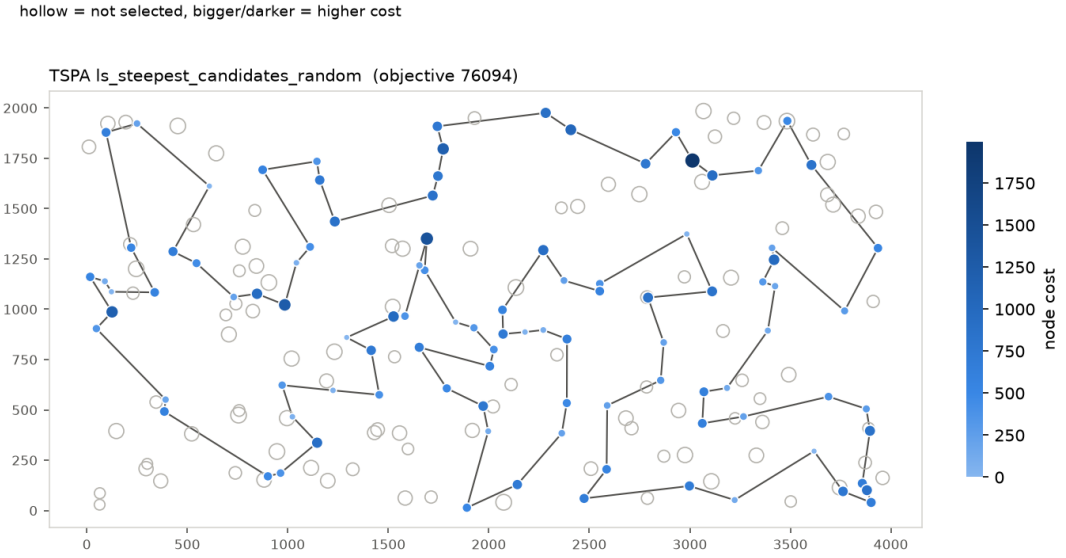
c	TSPA objective	TSPA time [ms]	TSPB objective	TSPB time [ms]
5	86663.6 (81042 – 92580)	67.41	54140.4 (51345 – 57164)	68.61
10	80258.1 (76094 – 84534)	143.32	52676.1 (50397 – 56560)	145.36
15	77493.8 (74743 – 82125)	222.87	52509.2 (50483 – 56035)	221.30
20	76621.4 (74282 – 80063)	303.62	52496.4 (50165 – 56035)	300.71
baseline	76184.6 (73961 – 80047)	726.96	52685.1 (50411 – 56416)	724.35

4 Visualization of the best solutions

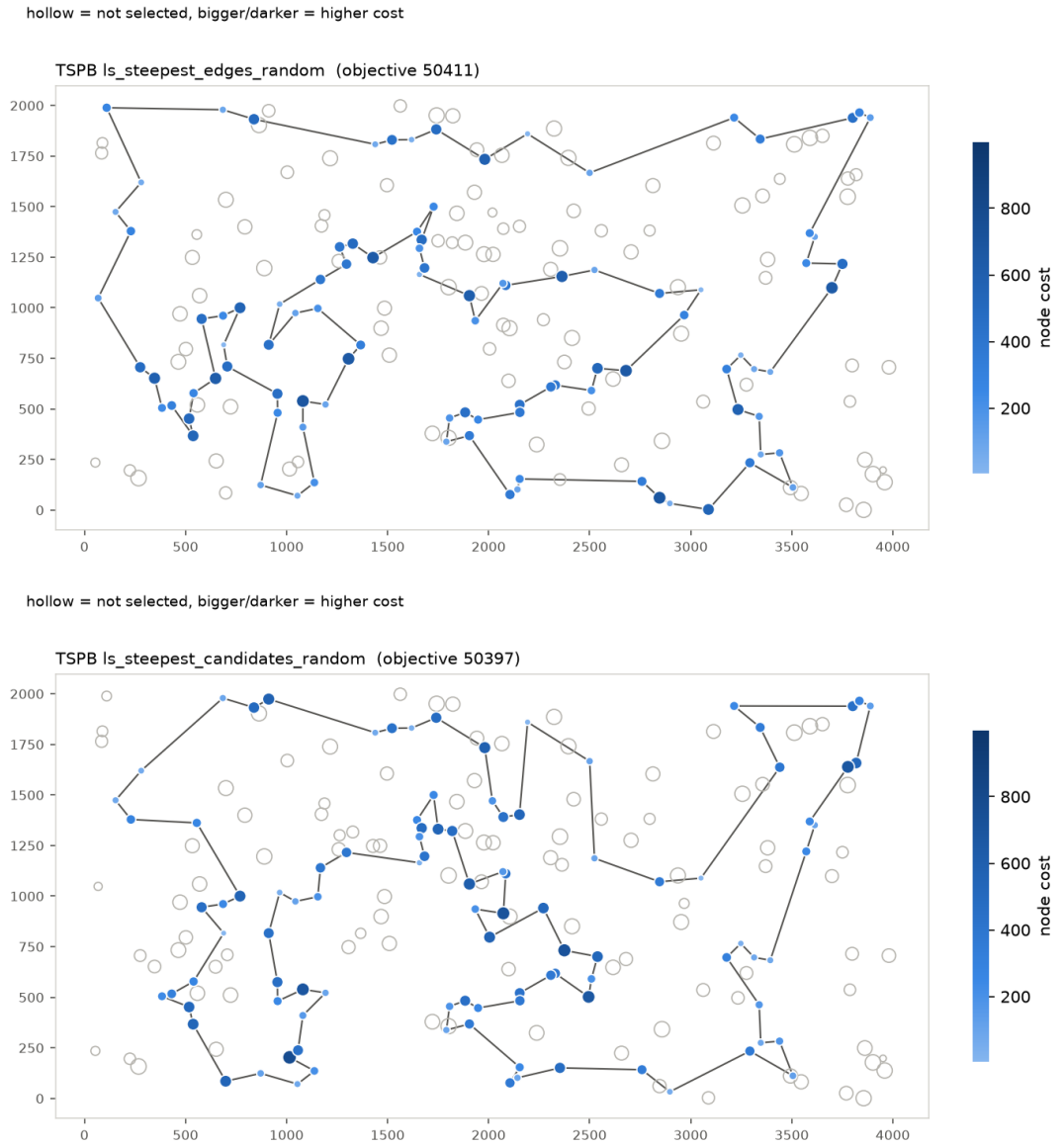
Each panel shows the best solution of one method. Selected nodes are filled, and both their size and color show the node cost (bigger and darker means more expensive). Nodes that are not selected are drawn as hollow grey circles, with size still showing their cost. Plots of the previous methods are in the previous reports.

TSPA


hollow = not selected, bigger/darker = higher cost
TSPA ls_steepest_edges_random (objective 73961)
node cost


hollow = not selected, bigger/darker = higher cost
TSPA ls_steepest_candidates_random (objective 76094)
node cost

TSPB



5 Best solutions

Lists of node indices, starting from 0, in the order they are visited. The cycle closes from the last node back to the first. Best solutions of the previous methods are listed in the previous reports.

TSPA

Baseline: LS steepest, two-edges, random start (objective 73961)

```
127, 161, 30, 187, 171, 2, 189, 141, 87, 98, 135, 73, 45, 140, 53, 179, 177, 36, 131, 84,
58, 192, 158, 134, 72, 151, 182, 57, 64, 88, 78, 193, 153, 75, 170, 146, 159, 107, 89,
106, 150, 176, 174, 85, 1, 169, 3, 50, 93, 130, 47, 147, 101, 29, 178, 184, 136, 186,
164, 142, 8, 54, 9, 33, 143, 52, 144, 67, 16, 51, 156, 35, 185, 102, 28, 180, 132, 76, 4,
71, 157, 91, 48, 31, 80, 38, 100, 123, 160, 152, 108, 49, 60, 116, 149, 63, 119, 168, 95,
97
```

LS steepest, candidate moves, random start (objective 76094)

```
176, 134, 158, 192, 182, 57, 107, 88, 78, 193, 153, 170, 159, 130, 89, 151, 150, 106, 93,
50, 3, 169, 1, 47, 147, 101, 29, 178, 184, 195, 136, 186, 154, 39, 115, 5, 112, 139, 4,
71, 157, 91, 76, 132, 162, 90, 123, 152, 108, 80, 48, 31, 60, 49, 63, 149, 116, 119, 168,
97, 127, 171, 2, 189, 187, 30, 141, 87, 28, 180, 156, 35, 51, 133, 16, 67, 144, 53, 140,
185, 45, 73, 135, 114, 181, 58, 84, 131, 36, 177, 179, 52, 143, 33, 54, 9, 164, 85, 8,
174
```

TSPB

Baseline: LS steepest, two-edges, random start (objective 50411)

```
197, 36, 29, 132, 103, 178, 65, 145, 28, 149, 37, 116, 147, 121, 198, 96, 48, 156, 89,
57, 190, 70, 81, 94, 105, 196, 114, 10, 148, 143, 90, 129, 127, 80, 182, 153, 20, 14, 2,
74, 17, 171, 39, 99, 179, 87, 82, 51, 85, 59, 73, 167, 181, 152, 34, 24, 67, 66, 84, 192,
54, 162, 76, 135, 3, 184, 151, 122, 69, 160, 172, 136, 120, 133, 189, 183, 30, 21, 38,
154, 97, 140, 47, 64, 33, 40, 141, 60, 45, 134, 175, 71, 155, 115, 170, 26, 125, 177,
180, 6
```

LS steepest, candidate moves, random start (objective 50397)

```
73, 59, 85, 51, 179, 82, 87, 88, 99, 171, 74, 2, 14, 20, 153, 80, 127, 129, 90, 10, 114,
196, 139, 61, 105, 94, 81, 190, 70, 187, 54, 162, 76, 57, 89, 86, 46, 72, 156, 48, 96,
198, 121, 42, 147, 116, 149, 28, 145, 158, 26, 125, 177, 170, 6, 29, 132, 197, 36, 62,
175, 134, 45, 142, 157, 60, 40, 141, 71, 155, 97, 154, 140, 47, 38, 21, 69, 160, 172,
136, 133, 120, 8, 117, 122, 184, 3, 111, 151, 98, 27, 11, 66, 67, 31, 24, 34, 152, 181,
167
```

6 Solution checker

All four best solutions were checked with the provided solution checker (`Solution checker.xlsx`), and every objective value matches. Our code additionally checks every generated solution for the number of nodes and for repeated nodes.

Before the experiment we also tested the implementation: for 20 random solutions every generated candidate move (39 612 moves) was applied, and in all cases its delta was equal to the difference of the full objective values, the move added at least one candidate edge, and *pos* stayed consistent with P. With $c = n - 1$ (every edge is a candidate edge) the result of the search had no improving move in the full neighborhood of Lab 3.

Instance	Method	Our objective	Checker objective	Match
TSPA	Baseline (steepest, two-edges)	73961	73961	yes
TSPA	Steepest, candidate moves	76094	76094	yes
TSPB	Baseline (steepest, two-edges)	50411	50411	yes
TSPB	Steepest, candidate moves	50397	50397	yes

7 Conclusions

- Candidate moves make the steepest local search about 5 times faster: 142.7 ms instead of 727.0 ms per run on TSPA and 145.4 ms instead of 724.4 ms on TSPB. The number of evaluated moves drops 8.5–8.7 times (about 230 000 instead of 1 990 000 per run), because an iteration evaluates at most 2 000 candidate moves instead of 14 850. The time drops less than the

number of moves, since generating the candidate moves (the loops over the lists and the look-ups in *pos*) also costs time.

- The number of iterations hardly changes (125–132 vs. 133), so the search makes about the same number of steps, each of them much cheaper.
- On TSPB the quality is the same as without candidate moves (average 52 676.1 vs. 52 685.1), and the candidate version even found a slightly better best solution (50 397 vs. 50 411). Its average is the best of all methods so far on TSPB.
- On TSPA the quality is worse: the average grows by 5.3% (80 258.1 vs. 76 184.6). The node costs of TSPA are twice as large as those of TSPB (up to 1 997 vs. 999), so they dominate the score $D[u][v] + cost[v]$ and the 10 "nearest" nodes are often cheap but distant nodes, while short edges are left out. Indeed, 9 of the 100 edges of the best baseline solution of TSPA are not candidate edges for $c = 10$, compared with only 2 for TSPB. The search cannot add such edges, so it stops earlier in worse local optima.
- The number of candidate neighbors trades time for quality. The time grows roughly linearly with c (about 68, 144, 222 and 302 ms for $c = 5, 10, 15, 20$). With $c = 20$ the TSPA average (76 621.4) is only 0.6% above the baseline and the search is still 2.4 times faster; on TSPB already $c = 10$ is enough, and $c = 15$ or 20 is slightly better than the baseline. Too few candidates ($c = 5$) clearly hurt on both instances.